

NYCU 2025 DCS 數位電路與系統 期中考

考試時間: 2025/04/10 16:30~17:20

姓名：

學號：

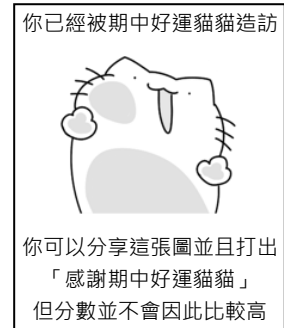
工作站帳號：

1. (25%) Block diagram.

For (a) and (b), please draw the block diagrams of the following two circuits using the symbols of adders, multiplexers, comparators, and D flip-flops.

※ comparator, multiplexer 之訊號順序請標示清楚。

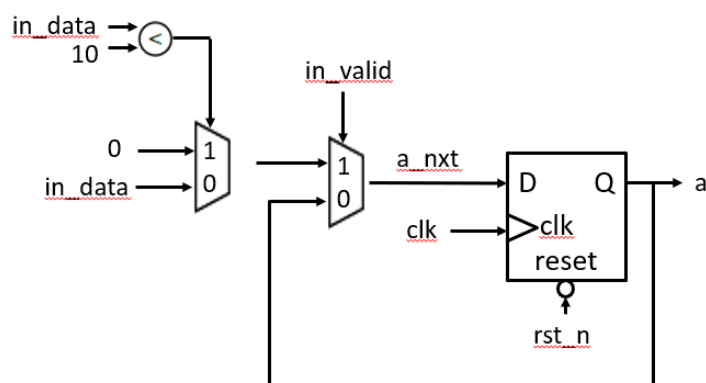
※ 所有訊號不論 multi-bit bus 或 1-bit signal 都用一條線表示即可。



(a) (15%)

```
always_ff @ (posedge clk, negedge rst_n) begin
    if (~rst_n)
        a <= 0;
    else
        a <= a_nxt;
end
always_comb begin
    a_nxt = a;
    if (in_valid) begin
        if(in_data < 10)
            a_nxt = 0;
        else
            a_nxt = in_data;
    end
end
end
```

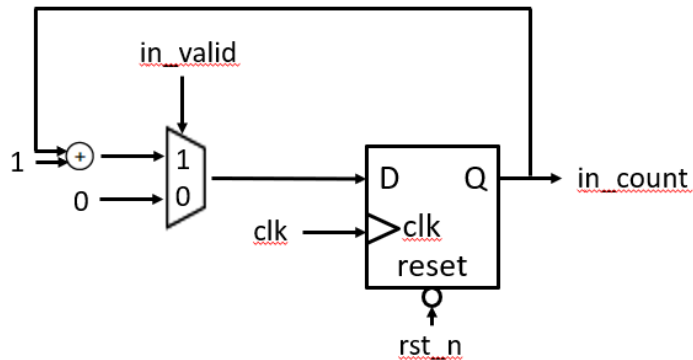
ANS.



(b) (10%)

```
always_ff @ (posedge clk, negedge rst_n) begin
    if (~rst_n)
        in_count <= 0;
    else if (in_valid)
        in_count <= in_count+1;
    else
        in_count <= 0;
end
```

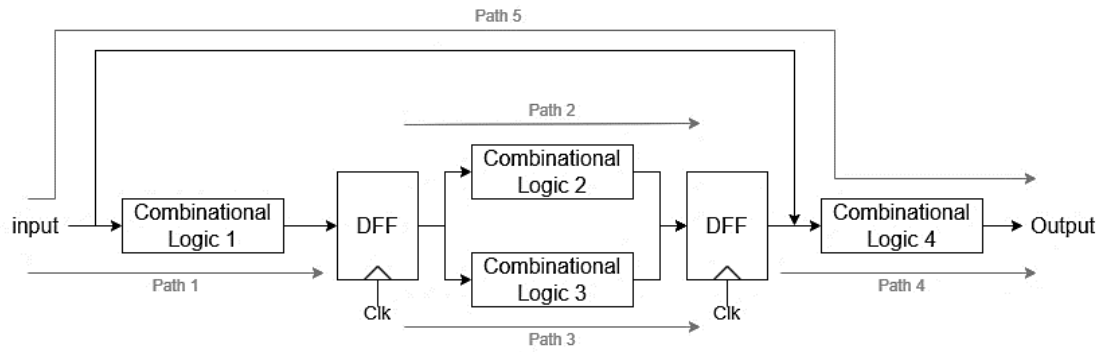
ANS.



評分標準:

1. 訊號名稱沒有標示和 Block 沒有標示清楚等一處扣 1 分
2. `rst_n` 畫成 synchronous reset 扣 4 分
3. FF 沒畫扣 6 分
4. Mux 多或少一個或邏輯錯誤扣 4 分
5. 沒畫 feedback path 扣 3 分
6. Multiple drivers 扣 2 分
7. 其餘依情況斟酌扣分

2. (25%) There is a circuit shown below:



Assume the clock cycle time is 5 ns. All D flip-flops change their values only at the positive edge of the clock. **The input and output signals have an 0.5*cycle external delay.** The delays for combinational logic blocks 1, 2, 3, and 4 are 3 ns, 2 ns, 7 ns, and 1 ns, respectively.

1. (10%) Please describe the problems you may face in 01_RTL and 02_SYN, and explain why or why not.

ANS:

Timing Violation Path: Path1, 3, 5

Path1: $2.5 + 3\text{ns} > 5\text{ns}$ Path2: $2\text{ns} < 5\text{ns}$ Path3: $7\text{ns} > 5\text{ns}$

Path4: $1 + 2.5\text{ns} < 5\text{ns}$ Path5: $2.5 + 1 + 2.5\text{ns} > 5\text{ns}$

01_RTL 只驗證 function 的正確性，並沒有考量時間，所以不會出錯。

02_SYN 會考慮時間，若 critical path 超出 cycle time，這樣 slack 就會 violated
評分標準：

(1) 01_RTL 寫出原因沒有錯誤得 5 分，如果寫 comb2, 3 multdrive 也給分

(2) 02_SYN 寫出 timing 問題得 5 分

(3) 如果 02_SYN 有寫 Path 但沒寫滿 Path1,3,5 扣 1 分

2. (15%) Propose three methods to fix the problems and indicate which path's problem can be fixed by each method (You don't need to fix all the path problems with one method).

ANS:

方法範例:

改演算法減少單一 cycle 內的運算，Path 1,3

切 pipeline，Path 1,3,5

用 register 把 input (output) 存起來，Path 1,5

提高 clock cycle time，Path 1,3

評分標準：

(1) 三個方法每個方法 5 分，其中方法說明 3 分，path 2 分

(2) Path 如果寫多個且有部分錯誤扣 1 分

(3) 如果在(a)有寫 multdrive，在(b)提出 multdrive 的解決方案也給分

3. (24%) Please find out 6 errors in the design below (i.e. the errors that cause 01_RTL and 02_SYN failed). Write down the line number of each error and briefly describe the type of error. Assume that both the input external delay and output external delay are equal to half of the clock cycle. (There are 6 errors in total, find all of them to receive full points)

Note: Errors of the same type can only be counted as one!!!

```

13  input clk, rst_n;
14  input in_valid;
15  input [4:0] in_data;
16  output logic [5:0] out_data;
17
18  parameter S_IDLE=0, S_INPUT=1, S_COMP1=2, S_COMP2=3, S_OUT=4;
19  logic [2:0] state, next_state;
20  logic [4:0] counter, counter_next;
21  logic [5:0] data1, data2, data3, data4, data1_next;
22
23  always_ff @(posedge clk or negedge rst_n) begin
24      if(!rst_n) begin
25          state <= S_IDLE;
26          counter <= 0;
27          data1 <= 0;
28      end
29      else begin
30          state = next_state;
31          counter = counter_next;
32          data1 = data1_next;
33      end
34  end
35
36  always_comb begin
37      case(state)
38          S_IDLE: next_state = (in_valid)? S_INPUT: S_IDLE;
39          S_INPUT: next_state = (data1 > 4)? S_COMP1: S_COMP2;
40          S_COMP1: next_state = (counter == 8)? S_OUT: S_COMP1;
41          S_COMP2: next_state = (counter == 3)? S_OUT: S_COMP2;
42          S_OUT: next_state = S_IDLE;
43      endcase
44
45      assign counter_next = (state == next_state)? (counter + 1): 0;
46      assign data1_next = (in_valid)? (in_data + 1): data1;
47  end
48
49  always_comb begin
50      data1 = in_data + 1;
51      data2 = (state == S_COMP1)? data1+2: data4;
52      data3 = data2 >> 1;
53      data4 = (state == S_COMP1)? data3-1: 0;
54      out_data = (state == S_OUT)? data4: in_data;
55  end

```

Ans.

行數	原因	答對率
Line 30~33	blocking, non-blocking assignment 混用	97.35 %
Line 43	case 沒有 default (latch inference)	88.50 %
Line 45~46	always 裡面不能 assign	91.15 %
Line 32, 50	data1 multiple driven	90.27 %
Line 51~53	combinational loop (data2, data3, data4) 寫 data4 → data2 時出現 unknown 也算對	28.32 %
Line 54	input 到 output 沒經過 flip-flop (negative slack)	41.59 %

評分標準：

每找出一個 error 得 4 分，沒寫行數或沒寫原因或行數錯誤不給分，其餘小錯誤斟酌扣分。以下常見答案不算 error：

- (a) 沒寫 module 和 endmodule
- (b) Signal bit mismatch
- (c) Signal overflow
- (d) 沒有 reset combinational signal
- (e) 第 51, 53 行的(data1+2), (data3-1)少括號

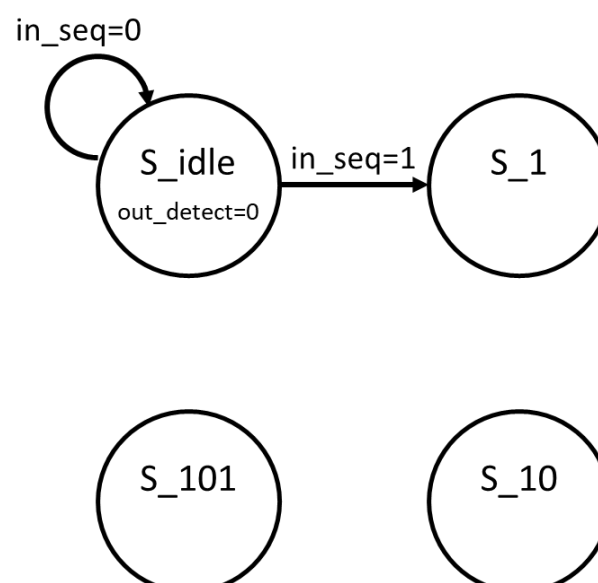
4. (26%) The following is a sequence detector design. It takes in_seq as an input signal, which is a stream of binary values (1s and 0s). When the specific pattern "101" appears consecutively in the input stream, the detector asserts the out_detect output with a pulse.

```

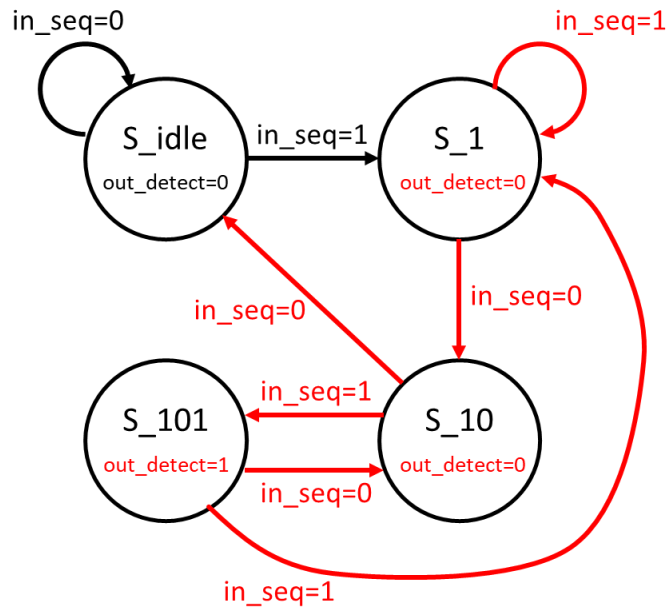
module SD(clk, rst_n, in_seq, out_detect);
input clk, rst_n, in_seq;
output logic out_detect;
logic [1:0] state_cs, state_ns;
logic in_seq_is_1;
parameter S_idle = 0,
           S_1    = 1,
           S_10   = 2,
           S_101  = 3;
always_ff @ (posedge clk or negedge rst_n) begin
    if(!rst_n) state_cs <= S_idle;
    else state_cs <= state_ns;
end
always_comb begin
    case(state_cs)
        S_idle: state_ns = (in_seq_is_1) ? S_1 : S_idle;
        S_1:    state_ns = (in_seq_is_1) ? S_1 : S_10;
        S_10:   state_ns = (in_seq_is_1) ? S_101 : S_idle;
        S_101:  state_ns = (in_seq_is_1) ? S_1 : S_10;
        default: state_ns = 'bx;
    endcase
end
assign in_seq_is_1 = (in_seq == 1'b1);
assign out_detect = (state_cs == S_101);
endmodule

```

- (a) (13%) This module is implemented using a finite state machine (FSM). Complete the state diagram below by clearly indicating all state transition conditions and the output associated with each state.



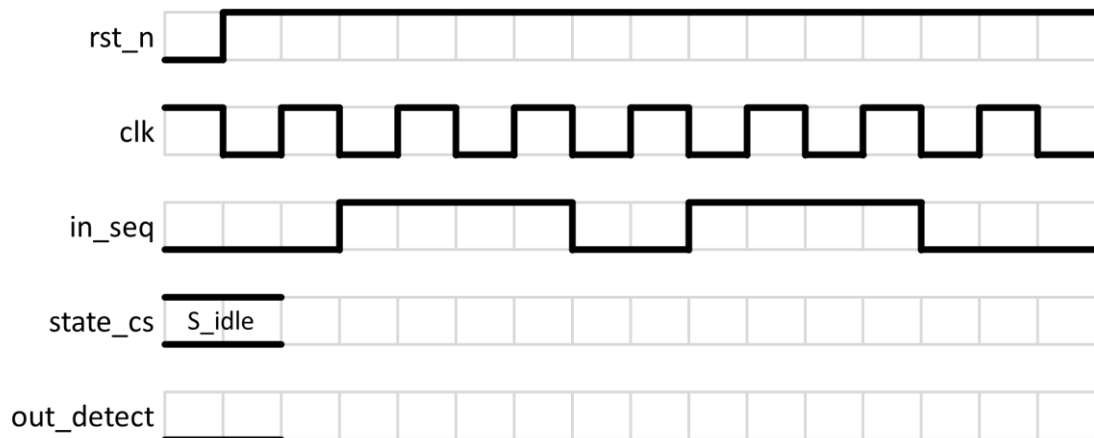
ANS.



每個箭頭、輸出，畫錯/多畫/少畫扣 2 分，扣完為止。

若畫上 reset 條件，不扣分也不加分。

- (b) (13%) Based on the input bitstream provided in the waveform below, complete the waveforms for the state_cs and out_detect signals. You should use the states S_idle, S_1, S_10, and S_101 to determine the values of state_cs.



ANS.



每個 clock cycle 的訊號(以 clk posedge 為分隔)畫錯扣 1 分，扣完為止。

若在 clk negedge 改變訊號，state_cs 視為全錯。